
CMP6200

Individual Undergraduate Project (FYP)

Final Report

**[Reinforcement Learning to improve AI
behaviour and adaptability in a game
environment]**

Student Name: Tamer Hussen

Student ID: S23130437

Course: Computer Games Technology

Supervisor: Jan Krasniewicz

Date: 5/5/26

Abstract

The project investigates the use of reinforcement learning (RL) to produce adaptive, believable non player characters (NPCs) within a unity game environment. Using unity ML-Agents, the work involves designing controlled training environments, implementing RL agents using proximal policy optimization (PPO), and comparing their behaviour against a custom C# scripted behaviour tree baseline utilising A* NavMesh pathfinding. The artefact is a Predator and Prey arena featuring an AI agent equipped with complex sensory systems, including a vision cone, hearing, and a multipoint scent trail. Training evaluations, capped at 20 million steps, revealed that while early-stage learning produces coherent tracking behaviours, extended training can exhibit instability and low reward plateaus. By introducing distraction animals, dynamic prey evasion tactics and a strategic familiar summoning system, the model successfully comes together. Outcomes demonstrate the critical importance of sensory integrity and reward shaping in practical RL, showing the trade-offs between designer control, and developing adaptability in playable game development.

List of Context

Abstract	2
List of Figures	4
List of Tables.....	5
1.0 Introduction and Problem Statement.....	6
1.1 Introduction and Context	6
1.2 Problem Statement.....	7
2.0 Literature Review	7
2.1 Review of Existing Knowledge	7
2.2 Critical Analysis and Gap Identification	8
2.3 Project Justification	9
3.0 Project Aims, Objectives, and Scope	10
3.1 Project Aim	10
3.2 Project Objectives	10
3.3 Project Scope	11
4. System Design and Architecture	12
4.1 Design and Methods	12
System architecture and Environment Design:	14
Algorithm implementation and training regime:	14
Validation and evaluation methods:	14
4.2 Justification of Methods	16
4.3 Environment Architecture and Player Systems	17
4.4 AI Sensory Systems Implementation	20
4.5 Reinforcement Learning Setup (ML-Agent & PPO)	21
5. Implementation (Unity, ML Agents, YAML)	23
5.1 The Behaviour Tree Baseline	23
5.2 Construction of the Artefact	24
5.3 Model Export and Inference	26
6. Experimental Method	27
6.1 Validation and Testing.....	27

7. Results	28
8. Comparative Analysis	31
8.1 Critical Evaluation.....	31
8.2 Analysis of Tester feedback and Exploits	33
9. Discussion	34
9.1 Limitations	34
9.2 Impact	36
10. Conclusion and Future Work.....	36
10.1 Conclusion	36
10.2 Future Work	37
References	38
List of Assets used in implementation	40
Appendices	41

List of Figures

Figure 1: Pathfinding.....	12
Figure 2: RL agent pipeline	13
Figure 3: Reward shaping schematic	13
Figure 4: GitHub Kanban board shows the transition from technical development phase to the final evaluation and reporting phase	15
Figure 5: Previous Gantt Chart	15
Figure 6: New Gantt Chart	16
Figure 7: Game Variables	18
Figure 8: Top-Down view of Arena	19
Figure 9: First person view.....	19
Figure 10: Training scene	21
Figure 11: Mean Reward progression across the 20 million step curriculum	29
Figure 12: Player Enemy Preference	32
Figure 13: Perceived Realism Comparison	32
Figure 14: YAML	41
Figure 15: ML Agent - Observation and Reward System.....	42
Figure 16: BT Agent – BT Structure and Nodes.....	43

List of Tables

Table 1: Technical Performance	29
Table 2: Tester Results.....	30
Table 3 : Risk Assessment	44

1.0 Introduction and Problem Statement

1.1 Introduction and Context

The project investigates how Reinforcement Learning (RL) can make artificial intelligence (AI) agents more adaptive and capable in a video game context.

Reinforcement Learning is an area of machine learning in which systems learn from experience, rewarding them for advantageous behaviours and punishing them for disadvantageous behaviours, rather than relying on prescribed rules (Sutton & Barto, 2018; MLVU, 2022).

In modern game development, most AI or NPC systems use some form of pre scripting or logic based on rules, which can lead to the AI agent becoming more predictable once the player observes the patterns built into the system (Tang, 2023; Isbister & Nass, 2000). By using RL, an AI agent can learn and improve while interacting with an environment, resulting in a more engaging, realistic, and replayable experience for the player (Mnih, et al., 2015; Silver, et al., 2017).

The project uses Unity's ML-Agents toolkit to achieve machine learning based AI. ML-Agents enables reinforcement learning directly within Unity using C#, allowing agents to interact with the game environment, receive feedback and adapt their behaviour through repeated simulations (Juliani, et al., 2018). This makes it possible to visualise and assess learning outcomes in real time, providing a practical and efficient way to develop intelligent, reactive game agents without relying on external Python integrations.

The main goal of the project is to develop and promote RL-based systems to those interested in bridging the gap between RL focused academic research and practical applications of game AI. By comparing an RL driven agent against a scripted AI agent, the project demonstrates that RL is much more advanced as a means of achieving behavioural complexity and realism in games (Senanayake, 2025). This practice is a huge area of commitment in modern games, where the adaptive capabilities of AI agents are of increasing importance for maintaining competition, experiencing immersion, and creating challenges for players.

1.2 Problem Statement

Normally, AI systems for games are fixed and predictable, basing their behaviours upon pre-defined scripts or decision trees that are unable to learn and adapt to the changing behaviour of the player (Glavin & Madden, 2018). This lack of adaptable AI limits player engagement and replayability.

To address this issue, the research examines how Reinforcement Learning can promote a more adaptive, self-learning AI that alters its behaviour through the feedback it absorbs from environment within Unity. Specifically, the research addresses the lack of comparative evaluations between modern RL algorithms and traditional highly optimised AI designs. The project aims to develop an AI that learns and evolves while the game is being played, ultimately comparing its efficiency, adaptability, and believability against a traditional Behaviour Tree baseline to see if learned logic can outperform rigid, pre-defined pathfinding (Lample & Chaplot, 2017; Isbister & Nass, 2000).

2.0 Literature Review

2.1 Review of Existing Knowledge

The integration of artificial intelligence in video games has historically relied on pre-scripted, deterministic models. Traditional game AI techniques such as Finite State Machines (FSMs) and Behaviour Trees (BT) are widely considered simple and effective solutions for generating non player character (NPC) logic. Behaviour Trees, in particular, provide an organised hierarchical structure of sequences, selectors, and conditional that dictate optimal pathing, often utilising algorithms and highly efficient, they lack true adaptability. They execute precisely what is programmed, which can result in predictable and rigid gameplay once players learn the underlying patterns (Tang, 2023).

In contrast, Reinforcement Learning (RL) enables continuous learning and adaptive behaviour. RL is based on the interaction between three main components: the agent, the environment, and the reward system. An agent takes actions, the environment gives feedback on those actions, and rewards help guide future decisions. Over time, through trial and error the agent learns to maximise positive benefits (Datatonic, 2022).

The project focuses on implementing Proximal Policy Optimisation (PPO), a highly stable, sample efficient algorithm commonly used in continuous control tasks

(Schulman, et al., 2017). Frameworks such as Unity's ML-Agents tool kit have recently made it possible to integrate these complex machine learning models directly within a game environment (Juliani, et al., 2018). This allows developers to implement simulated training sessions where agents continuously improve navigation, pathfinding, and decision making over repeated trails.

Selected subtopics in reinforcement learning are particularly relevant for game AI and will frame the literature review and practical experiments. This includes reward shaping and curriculum learning, techniques that influence learning speed, prevent reward hacking, and gradually increase task complexity, and sensory processing to allow agents to filter out the irrelevant environmental noise, as well as the practical trade-offs between trial-and-error steering logic versus perfect pathfinding models.

Previous research has shown some potential for RL in applied fields, including robotics, autonomous vehicles, and simulation-based control, but research into its applications in interactive video games is minimal (Pan & Yang, 2010; Tang, 2023). Most of the RL work for video games falls under either experimental or non-playable simulation, meaning players are unable to see the benefits of the AI being capable of adaptive learning in real time alongside a human player. This review establishes that although the potential of RL is acknowledged, its practical use in commercial or playable game AI is lacking, which forms the core foundation of this project.

2.2 Critical Analysis and Gap Identification

Although Reinforcement Learning theory is studied and reviewed to a large extent (Pan & Yang, 2010), most of the studies tend to apply RL to controlled simulations rather than applying RL in gameplay environments. These studies fail to explore how learning affects player engagement or how it affects game balance or AI personality.

Most examples of training ML Agents to address these issues in a Unity compatible format focus primarily on driving completion of a task, such as simply getting an agent to move towards a goal, without analysing if learning positively impacts in game adaptability or responsiveness (Senanayake, 2025). Furthermore, there is a distinct lack of comparative evaluations between RL driven agents and traditional, highly optimised AI designs. While traditional Behaviour Trees use perfect mathematical pathfinding, ML agents must learn to steer via trial and error (Hester, et al., 2018), creating a potential for

evolving behaviours such as cutting off a player's path or flanking, that are not explicitly programmed (Jaderberg, et al., 2019).

The identified key gaps are:

- There is a lack of studies applying RL in fully playable , interactive game contexts.
- There is insufficient research evaluating the practical trade offs between the rigid predictability of Behaviour Trees and the developing adaptability of PPO agents when placed in identical, playable scenarios.
- Performance analysis for trained agents is uncommon beyond success rate, ignoring the impacts of complex reward design and environmental noise / distractions.
- There is little research evaluating player perception of RL based behaviour compared side by side with scripted baselines.

These gaps suggest the need for applied projects that demonstrate and measure how RL can make game AI smarter, adaptive, and more engaging.

2.3 Project Justification

The goal of the project is to address the previously identified gap by building a playable, Unity-based predator and prey environment designed to display and evaluate reinforcement learning in a realistic gaming setting (Senanayake, 2025). By developing two identical AI predator agents , one controlled by a significantly trained PPO ML-Agent and the other by a custom built, NavMesh driven Behaviour Tree, this project establishes a rigorous baseline for scientific comparison.

The project investigates how altering aspects of the reward system, environmental complexity, and sensory integrity affects the ML agent's learning speed and hunting behaviour. The environment includes complex sensory mechanics such as multi ray vision cone, a volume-based hearing system to distinguish between player footsteps and ambient noise, and a multipoint scent trail (Bontrager, et al., 2018; Zook, et al., 2019). It also explores how the agent adapts to environmental challenges, including soft obstacles, like bushes to mask scent, and moving distraction animals.

Comparing the PPO trained agent against a traditional scripted AI highlights the dynamic difference in learning, developing tactics and adaptability (Glavin & Madden, 2018). In

addition, a questionnaire captures feedback on how users perceive AI intelligence or realism, offering both technical and player-based feedback on believability (Isbister & Nass, 2000). The outcome contributes to a better understanding of how reinforcement learning can be practically used in real time game development, proving whether learned hunting styles are more effective and engaging than standard optimal pathing AI (Lample & Chaplot, 2017).

3.0 Project Aims, Objectives, and Scope

3.1 Project Aim

The aim of this project is to design, create and assess a reinforcement learning system within a Unity based 'Predator and Prey' game environment to evaluate whether an AI's adaptability, developing behaviour, and decision-making processes offer a more engaging and effective alternative to traditional scripted AI baselines (Senanayake, 2025).

3.2 Project Objectives

- **Objective 1:** Create a 3D 'Predator and Prey' Unity environment equipped with complex AI sensory systems, including a vision cone, hearing via a sound emitter system, and a multipoint scent trail (Khan, 2014).
- **Objective 2:** Implement and train a Proximal Policy Optimisation (PPO) agent using Unity's ML-Agents toolkit (Juliani, et al., 2018), using targeted reward shaping and multi-phase curriculum training capped at 20 million steps (Schulman, et al., 2017).
- **Objective 3:** Continuously improve the reward structure and environment design, such as introducing distraction animals and a familiar summoning system, to break low reward training periods and prevent aiming for the minimum (Bontrager, et al., 2018).
- **Objective 4:** Develop a custom, A* NavMesh driven Behaviour Tree(BT) baseline from scratch, mirroring the ML agent's speeds, recognition thresholds, and cooldowns to ensure a scientifically fair comparison (Glavin & Madden, 2018).

- **Objective 5:** Evaluate and compare the performance, learning rates, and developing behaviour of the RL agent against the rigid pathfinding of the scripted BT baseline within identical playable survival scenarios (Lample & Chaplot, 2017).
- **Objective 6:** Conduct an analysis of agent believability and tactical development, assessing whether the RL agent exhibits human like strategies such as flanking or ambushes compared to the scripted baseline (Isbister & Nass, 2000; Jaderberg, et al., 2019).

3.3 Project Scope

To ensure the project remained focused and achievable within the given time limit, clear boundaries were established.

Included:

- Integration of the Unity ML-Agent toolkit to facilitate reinforcement learning directly within the engine (Juliani, et al., 2018).
- Development of a specialised Predator and Prey simulated environment.
- Implementation of advanced sensory mechanics, such as sight, sound and scent masking, and environmental distractions, like wandering animals (Zook, et al., 2019).
- Creation of a custom C# Behaviour Tree for direct comparison.
- Visualisation of training progress via generated data logs, debug UIs, and ONNX model exports (Datatonic, 2022).
- Collection of feedback from users to evaluate perceived realism and difficulty.

Excluded:

- The 'Adaptive Maze Runner' sub goal, which was removed from the scope early in development to ensure the core Predator and Prey arena reached stability and complexity.
- Multiplayer or online networked systems.
- Physical robotics or real- world machine learning applications.
- Deep learning architectures require high end computing power outside of the standard ML-Agent framework.

4. System Design and Architecture

4.1 Design and Methods

The project used an iterative, Agile inspired software development methodology, prioritising rapid prototyping, continuous testing, and sequential complexity scaling. The core artefact was developed using the unity engine (C#), supported by the Unity ML-Agents toolkit to facilitate the machine learning environment (Juliani, et al., 2018).

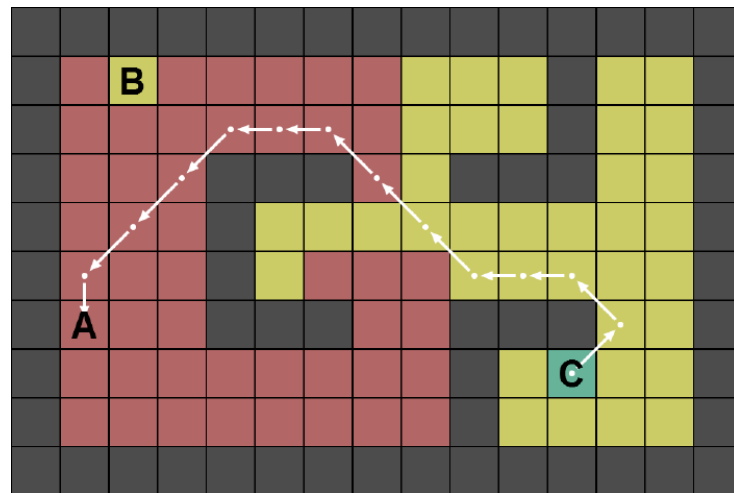


Figure 1: Pathfinding

This figure shows C uses pathfinding to avoid obstacles and reach A, while B is represented as an intermediate waypoint in the route (Khan, 2014).

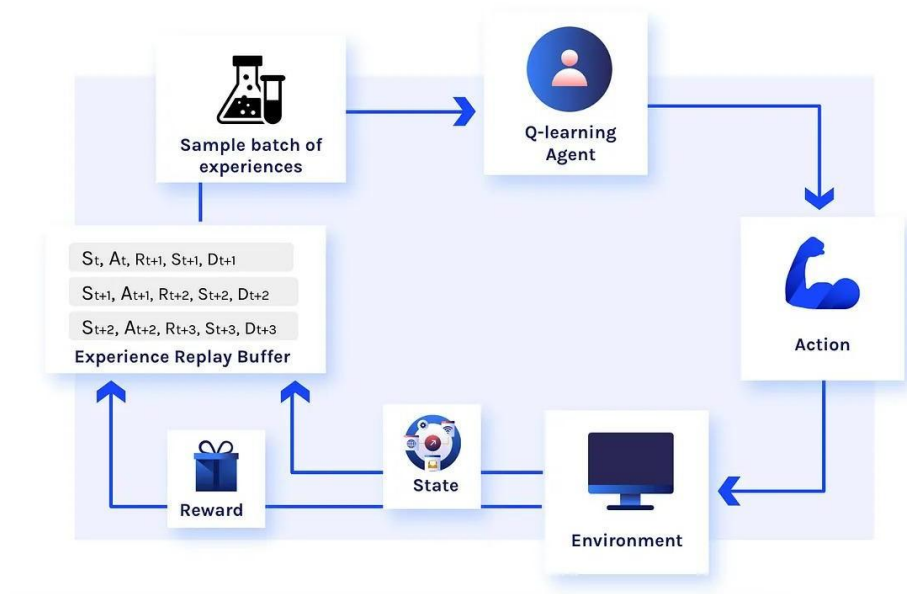


Figure 2: RL agent pipeline

This figure shows how a reinforcement learning framework for a Q- learning agent works(the first S is current state, A is current action, R is reward, next S is next state and D is the done signal) (Datatonic, 2022).

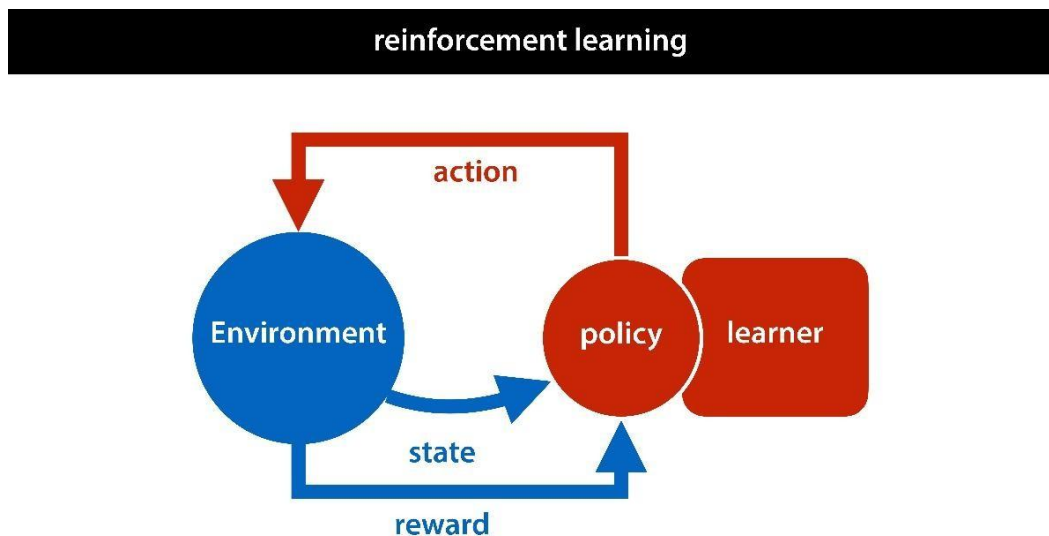


Figure 3: Reward shaping schematic

This figure shows how rewards can reinforce the agent to follow the policy and interact with the environment (MLVU, 2022).

The design is split into three primary methodological phases:

System architecture and Environment Design:

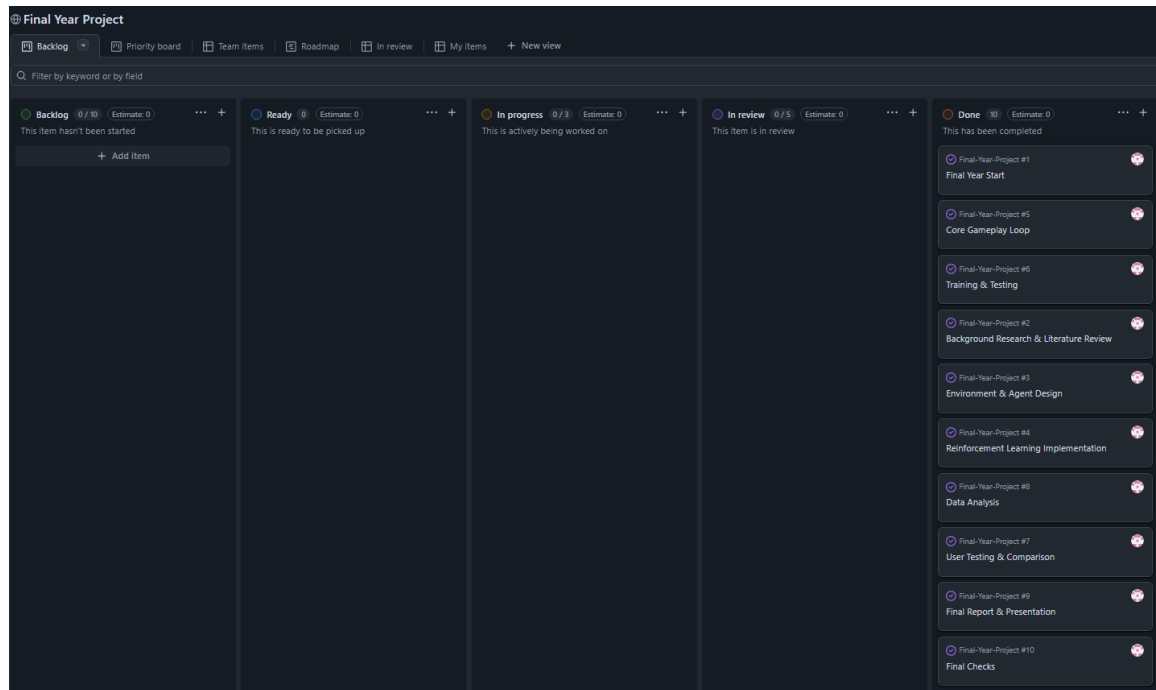
A 3D 'Predator and prey' survival arena was constructed featuring dynamic terrain, solid obstacles (like walls, trees, and rocks to break the line of sight) and soft obstacles (bushes to mask scent and block line of sight). The AI agents were equipped with complex multi layered sensory inputs: a multi ray vision cone, a volume-based hearing system capable of distinguishing sound sources (filtering player noise from ambient distraction animals) and a timed multipoint scent trail.

Algorithm implementation and training regime:

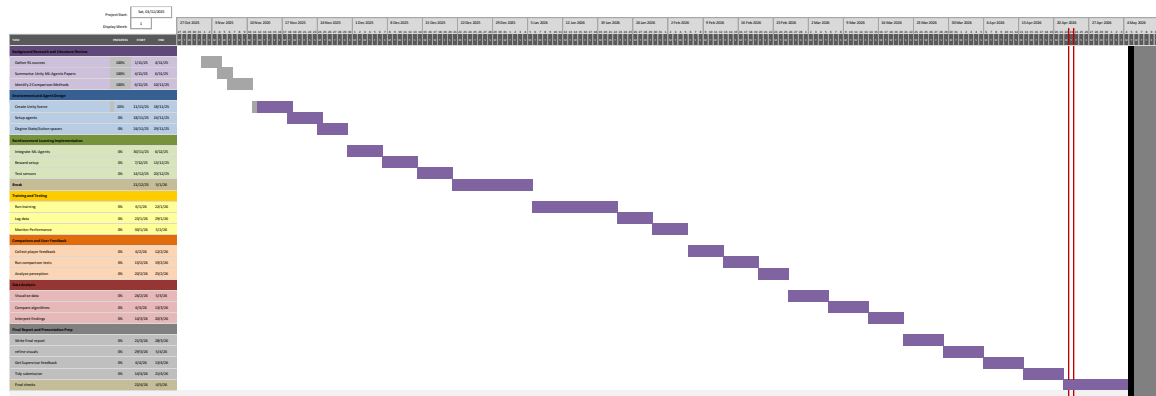
- The ML agent: A Proximal Policy Optimisation (PPO) algorithm was selected to drive the learning predator (Schulman, et al., 2017). The agent's neural network was configured using YAML files, feeding a 90-dimension observation vector into the model (Datatonic, 2022). Training was conducted over a rigorous 20 million step curriculum divided into sequential phases:
 - Phase 1: Establishing basic tracking and wall avoidance.
 - Phase 2: Introducing aerial and ground familiars to aid in hunting.
 - Phase 3: Adding distraction animals to evaluate sensory filtering and focus.
 - Phase 4 and 5: Fine tuning speed/ stamina mechanics and introducing scent making to break late-stage reward plateaus.
- The Baseline Agent: To establish a control group, a custom Behaviour Tree (BT) framework was architected entirely from scratch in C#. The BT uses A* pathfinding via Unity's NavMesh to navigate the arena optimally (Khan, 2014). Its decision-making hierarchy evaluates eight specific states per frame (e.g., strike, stalk, track scent, summon familiar), providing a rigid, mathematically flawless counterpart to the ML agent.

Validation and evaluation methods:

Evaluation relies on a side-by-side comparative methodology. Both the ML Agent and the BT baseline were contained to identical parameters, which were exact speeds, recognition thresholds, cooldowns, and stamina limits. Data logging and in game debug UIs were used to record quantitative metrics (for example detection distance, time to catch, and success rates), while user playtesting generated qualitative data on developing behaviours during playable survival scenarios.



GitHub Kanban Link



calculated paths without trial and error it serves as the perfect hard coded control (Glavin & Madden, 2018). Mirroring the exact speeds and constraints across both the BT and ML agent, specifically disabling the stamina drain to compensate for the ML agent lacking the built-in flawless pathfinding of the NavMesh, ensuring that the final evaluation is scientifically fair. This methodology effectively isolates the variable being evaluated, which is the ML agent's learned, evolving hunting style versus the rigid predictability of a human designed algorithm, thereby conclusively answering the project's core research question.

4.3 Environment Architecture and Player Systems

The simulation environment was constructed in Unity, using imported 3D assets (ALP and Nobiax/Yughues) to create a rocky, walled off survival arena. The arena features hard obstacles like walls and rocks to break the line of sight, and soft obstacles like bushes that are used to mask the players scent. A first-person player controller was implemented to function as the prey.

Based on pilot testing and iterative balancing, several player centric systems were refined to ensure fairness and atmospheric tension during evaluation:

- Audio and visual cues: A dynamic audio fade system and an expanded dark vignette effect were implemented to warn the player visually and audibly as the predator approaches, creating a sense of dread before physical contact.
- Control refinements: default mouse sensitivity was optimised, and a live UI sensitive slider was added to ensure human testers had precise control during evasion manoeuvres.

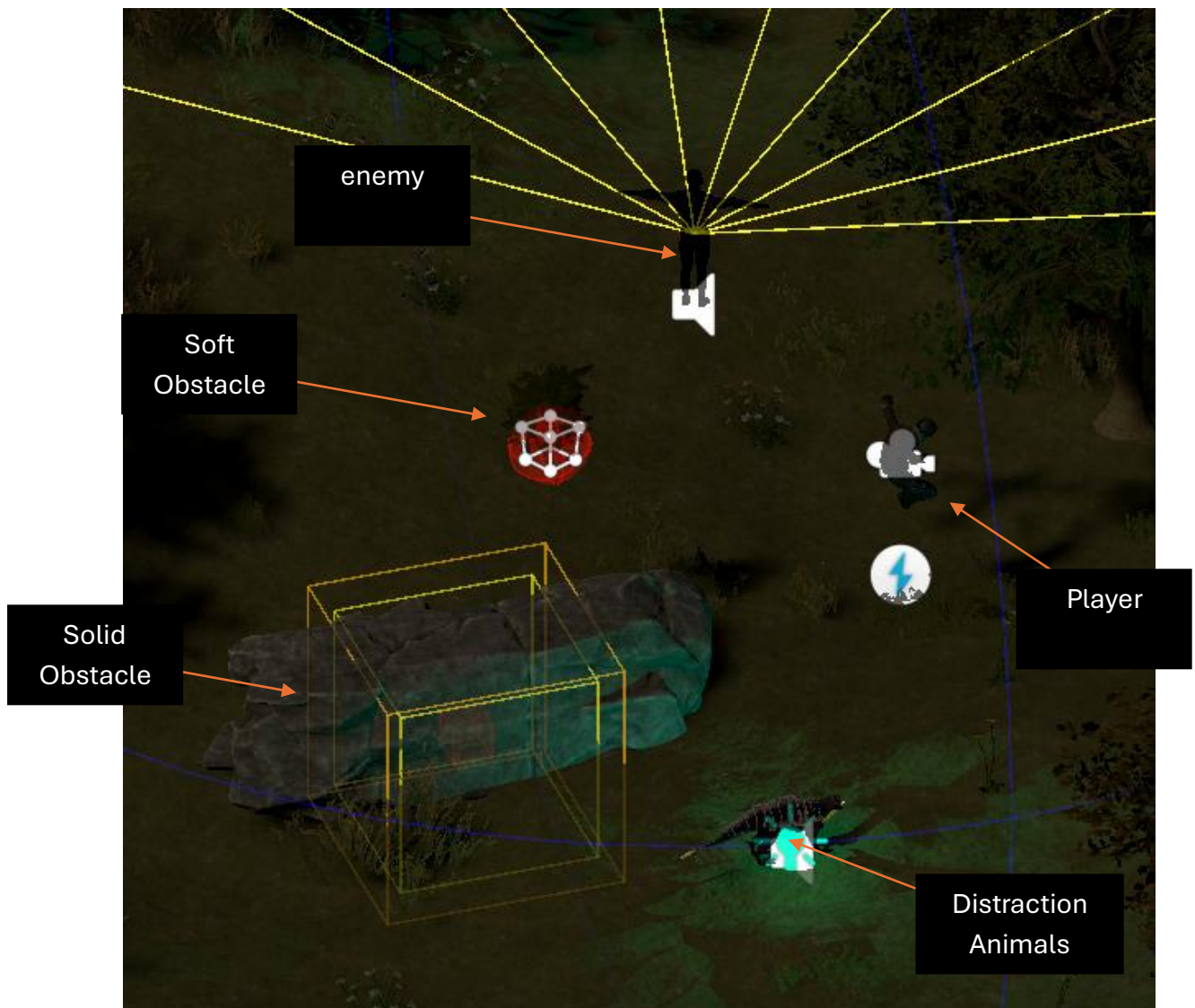


Figure 7: Game Variables



Figure 8: Top-Down view of Arena



Figure 9: First person view

4.4 AI Sensory Systems Implementation

To ensure the ML agent had sufficient data to learn complex tracking rather than relying on random exploration, a 90-dimension observation vector was established (Hester, et al., 2018). This vector is fed by three primary custom built sensory mechanisms:

- Visual system: A multi ray cast vision cone detects line of sight collisions with the player dynamically updating the agent's target tracking when the player is not obscured by solid obstacles (Khan, 2014).
- Hearing system: A volume-based audio detection mechanism allows the agent to perceive sound emitters. Crucially, this system is programmed to distinguish between the footsteps of the player and the ambient noise generated by the newly introduced distraction animals, forcing the agent to learn to filter out irrelevant audio cues (Jaderberg, et al., 2019).
- Multipoint scent trail: A time limited trail of scent markers is left behind by the player. The agent detects these nodes to track the player's historical path. If the player enters a designated soft obstacle like a bush, the scent emission is temporarily masked, forcing the agent to rely on visual or audio search patterns.

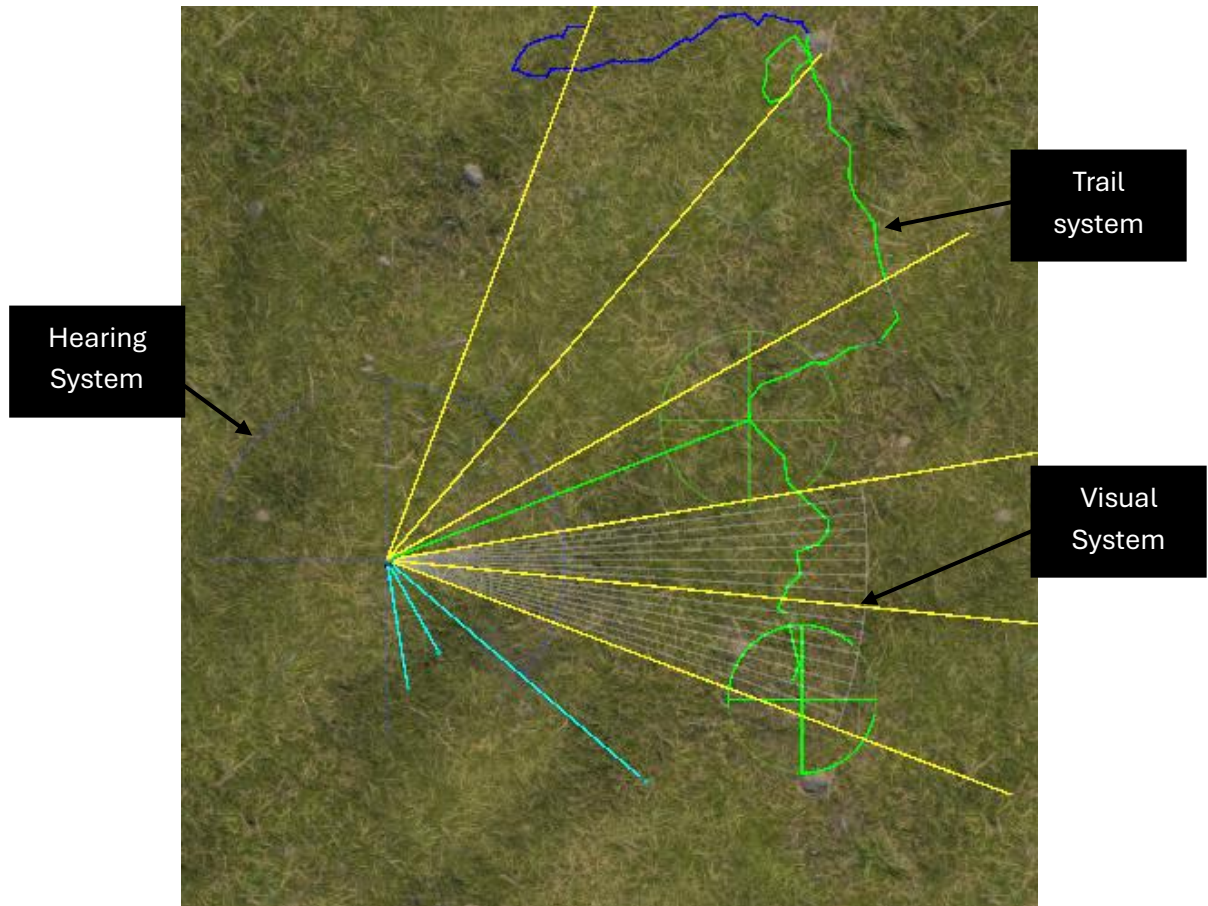


Figure 10: Training scene

4.5 Reinforcement Learning Setup (ML-Agent & PPO)

The predator's learning was driven by the Proximal Policy Optimisation (PPO) algorithm via Unity's ML-Agents package (Juliani, et al., 2018; Schulman, et al., 2017). The agent's neural network and training hyperparameters were configured using a YAML file (see Appendix A for full YAML code), carefully balancing the learning rate, batch size, and buffer size to maintain stability over extended runs (MLVU, 2022; Datatonic, 2022).

- **Network Capacity:** To manage the complex sensory system, the network capacity had the original 256 hidden units and 2 layers increased to 512 and 3 layers. The starting numbers, such as 256 units, buffer size 81920, batch size 1024, and beta 1.0e-3, were the default recommended parameters provided by the ML-

Agents toolkit for basic training. These default settings were increased because the agent needed more capacity to process the complex sensory data in the 3D environment. The decision to use 3 layers, even though it is not divisible by 512, was not based on strict calculations. Instead, it was the result of trial and error during testing to find a set up that worked and allowed the agent to learn. A memory variable was originally assessed but was removed after causing the training process to slow down.

- Fixing learning curve: when the early mean reward plateaued, the buffer size was reduced from 81920 to 20480 and the beta was increased from $1.0e-3$ to $5.0e-4$. This forced the agent to become more active in exploring the environment instead of staying with early bad habits.
- Late-stage stability: As the environment became more complex, the batch size was increased from 1024 to 2048 for smoother network updates. Also, the learning rate was lowered to $1.5e-4$ to secure the agent's behaviour and stop it from forgetting its progress.

Training was executed via the command line console over a massive 20 million step curriculum. The reward function was heavily shaped to encourage efficient hunting while penalising unreliable movements:

- Positive rewards: Granted for moving closer to the player, maintain the line of sight and successfully capturing the player.
- Negative rewards: Applied for colliding with solid obstacles, losing the scent trail, failing to capture the player within a set time limit, or exploiting reward loopholes (like spinning to artificially maintain line of sight).

Once a training phase stabilised, the resulting neural network was exported as an ONNX (open neural network exchange) checkpoint and assigned to the in-game agent for real time inference and testing.

5. Implementation (Unity, ML Agents, YAML)

5.1 The Behaviour Tree Baseline

To scientifically evaluate the ML agent's developing behaviour and establish a rigorous control group, a traditional Behaviour Tree (BT) was written entirely in C#. By writing this system from scratch instead of relying on third party plugins, total transparency and control over the decision-making logic of the control agent was achieved. This is particularly important in an academic comparison as it removes any outside abstraction layers that could otherwise hide how decisions are being made internally.

The BT uses Unity's built-in NavMesh system in combination with the A* pathfinding algorithm to generate the most efficient route to the player (Khan, 2014). The A* algorithm mathematically guarantees the shortest path by evaluating nodes based on the cost of the path to the current node plus an estimate of the cost to the target. This is represented by the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach the current node and $h(n)$ is the estimated cost to the goal. This deterministic nature ensures that the BT agent always follows the shortest possible path given the environmental constraints.

While this level of optimisation is beneficial for efficiency and predictability, it introduces limitations in how diverse the behaviour can be. Because the BT executes predefined logic without any deviation, it cannot produce unscripted behaviours beyond what has been explicitly programmed. This aligns with existing research suggesting that predictable AI systems, although dependable, often lack adaptability and long-term variety (Tang, 2023). In contrast, reinforcement learning systems may sacrifice this perfect pathfinding in favour of being more flexible.

The decision-making hierarchy functions as a prioritised stack that evaluates eight distinct states every frame, ensuring the agent reacts instantly to environmental changes. These states include:

- Headstart and Taunt: Beginning states designed to establish the predator's presence and allow the player a brief window to get their bearings.
- Strike and Stalk: High priority movements that activate when the player is within the agent's (predator's) line of sight, driving the core pursuit.

- **Track Scent and Track Sound:** Reactive states that activate when visual contact is broken but the agent can still sense the player, allowing for constant pressure even through obstacles.
- **Summon Familiar and Wander:** Low priority fallback state used to regain tactical awareness of the map.

The structured approach ensures stability and predictability but also reinforces the rigid nature of scripted AI. A critical aspect of this implementation was the enforcement of identical physical limits. To ensure the final comparative analysis focused purely on decision making and tracking logic, absolute similarity was maintained regarding physical constraints. Specifically, stamina drain for both agents was disabled. This is because the ML agent, unlike the BT agent, lacks the built-in flawless pathfinding of the NavMesh and instead relies on learned physics-based steering. Disabling stamina ensured that the ML agent was not unfairly penalised by the micro corrections during navigation. By isolating the independent variable, the internal validity of the experiment was strengthened, allowing for a fair evaluation of its strategic intelligence without any physical disadvantages (Glavin & Madden, 2018).

5.2 Construction of the Artefact

The enemy behaviour was inspired by the 1987 film *Predator*, specifically the way the alien hunts a team of elite soldiers one by one. The goal was to move beyond the aimless wanderer trope often seen in horror games and instead create an AI that feels like it is actively hunting its prey. To achieve this, the construction of the ML agent relied on a multi phased curriculum training approach, where task complexity is gradually increased over time (Sutton & Barto, 2018). This methodology is widely used in reinforcement learning to prevent agents from becoming overwhelmed by complex environments before mastering fundamental behaviours.

By sequencing the complexity of the tasks, this allowed the neural network to solidify foundational navigation skills in a simple environment before being overwhelmed by more complex sensory noise (Pan & Yang, 2010). By sequencing the complexity of the tasks, the agent could build its logic step by step. This training pipeline capped at 20 million steps, which was organised into the following critical phases:

- Phase 1 (0 – 2 million steps): The agent was placed in a clean environment, without any obstacles or distractions. This was done so it could master the basic physics, learn how to avoid walls, and practice following a simple scent trail. Starting in an empty space allowed the agent to build a strong foundation for its movement before things got more difficult. This aligns with the principle that early-stage learning should focus on foundational skills before introducing environmental noise.
- Phase 2 (2 – 5 million steps): The agent was given the ability to summon AI familiars, which included both ground and aerial scouts. This added a strategic layer to the agent's decision making. To stop the agent from relying on this too much through ability spamming, a small penalty of -0.1 every time a familiar was summoned (Bontrager, et al., 2018). Implementing this penalty ensured that the agent did not exploit the mechanic, which demonstrates the importance of reward shaping in preventing unintended behaviours. This highlights a key challenge in reinforcement learning, which are agents will often optimise for the highest reward rather than the developer's actual intention, requiring careful calibration of incentives. This forced the agent to think strategically, using the familiars as a backup resource only when the player's scent trail was completely lost.
- Phase 3,4 and 5 (5 – 20 million steps): Distraction animals were added to the map to create noise through sounds and movement. This taught the agent how to ignore irrelevant information and stay focused on the hunt. By forcing the agent to develop selective attention and distinguish between relevant and irrelevant stimuli, the agent learned to prioritise important signals rather than reacting to all inputs equally. In addition, a curiosity module was also added into the YAML configuration, which gave the agent a small reward for exploring new parts of the map (Burda, et al., 2018). This was an important step because it stopped the agent from getting stuck in a local minima, which is when an AI gets caught in a loop , like spinning in a circle or walking into walls because it cannot find a path.

The most significant change faced during construction was a severe low reward training plateau. At first, the predator caught the player far too quickly and efficiently . while this

might seem like a positive result, it actually meant the agent never had the chance to experience or learn more advanced stalking and cornering tactics. The agent had initially converged on a simple strategy of direct pursuit, which prevented further behavioural complexity from emerging.

To fix this issue, the movement variables for the player were dynamically adjusted and improved. The player's walking speed was increased to 2.75 and the sprinting speed to 5.5, while the predator's sprint was capped at a lower 5.2. This change allowed the player to outpace the predator in short bursts, making it impossible for the agent to win by simply running in a straight line. This adjustment demonstrates how environmental constraints can be used as a tool to shape learning outcomes. Instead of modifying the agent's brain directly, the problem space was changed to encourage more complex behaviours. Therefore, the ML agent was forced to abandon basic brute force chasing in favour of learning how to cut off corners and use predictive interception to catch the faster human target (Jaderberg, et al., 2019).

5.3 Model Export and Inference

Once the curriculum training reached the 20 million step milestone, the brain was exported from the Python environment as an ONNX (open neural network exchange) file (Juliani, et al., 2018). This file acts as a container for the finalised neural network, capturing the weights, biases, and all the learned logic gained over millions of training iterations. In essence, it represents the complete "wisdom" the agent developed during the curriculum process. This specific format is used because it allows for interoperability between different platforms, allowing the trained model to be deployed independently of the original training environment.

To integrate this intelligence into the final game, the ONNX model was assigned to the Behaviour Parameters component on the Predator prefab within Unity. By switching the system from default / heuristic mode, which is used for manual testing and training, to inference mode, the agent was able to make real time decisions directly inside the game engine. The transition from training to inference represents a significant shift in performance demands. During the training phase, the agent must continuously update its policy based on reward feedback, which requires considerable processing power. However, inference mode involves only forward passes through the neural network, making it significantly more efficient and suitable for real time applications.

This setup is important because it allows the agent to function as a standalone feature that is completely independent of the Python API. By integrating the ONNX model directly into Unity, the agent operates entirely within the game engine without having to rely on external system. This design ensures that the predator can run efficiently without any external lag or the need for an internet connection, providing players with a seamless and responsive encounter (Datatonic, 2022). Maintaining low latency and consistent performance in this way is critical for keeping the player immersed in a real time game environment.

6. Experimental Method

This project uses a mixed methods approach to evaluate the AI performance. While quantitative data gives a baseline for efficiency, it misses how players actually perceive the behaviour. In game AI, effective behaviour is often a matter of interpretation, so qualitative feedback is important to ensure the AI feels right in context.

6.1 Validation and Testing

To ensure that the ML agent functioned as intended and to allow for fair comparative testing, several frameworks and gameplay mechanics were established. Validation was not merely about checking if the AI worked but ensuring it provided a fair and balanced challenge for players. The following four systems were implemented to support this evaluation:

- Side-by-side debugging: A custom Debug UI was built to run both the ML agent and the BT agent simultaneously on screen. This UI allowed for the visual monitoring of the internal thoughts of both agents by overlaying cumulative reward graphs and line of sight (LOS) indicators directly onto the game view. Displaying internal states, rewards and distance to the player enabled real time monitoring of their decision-making processes, helping to confirm why an agent chose to stalk versus why it chose to strike (Juliani, et al., 2018; Datatonic, 2022).
- Player testing infrastructure: A reliable main menu was developed to help user testing. This infrastructure allowed human participants to choose to face either “Enemy A” (ML) or “Enemy B” (BT) without knowing which was which. The use of

blind testing ensures that participants are unaware of which AI system they are interacting with, which created a study environment that minimised bias toward the AI label. This ensures that the feedback was based on the gameplay experience rather than preconceived notions about machine learning (Isbister & Nass, 2000).

- **Objective Evaluation Metrics:** To move beyond purely subjective feedback, a strict 90 second survival timer and a timed escape win condition were added. Reaching the escape zone after this period served as a hard data point for success. The system meticulously tracked success states (whether the player escaped or was caught), the time to catch, and the detection distance, providing concrete data for the final comparison (Lample & Chaplot, 2017; Glavin & Madden, 2018).
- **Iterative bug resolution:** Validation testing revealed several technical friction points and critical bugs. One major issue was jittering movement caused by the ML agent's logic firing in the Update loop while physics calculations occurred in FixedUpdate. Additionally, the audio system would spam sound effects if the player briefly flickered in and out of the line of sight, and map randomiser occasionally spawned obstacles inside walls. These issues were systematically resolved to ensure smooth, uninterrupted gameplay, making sure that players were reacting to the AI's strategic intelligence rather than technical glitches (Bontrager, et al., 2018; Zook, et al., 2019).

7. Results

Testing successfully confirms that the artefact functions as a playable 'Predator and Prey' survival arena. The environment successfully integrates a complex RL agent capable of navigating 3D terrain while processing multi layered sensory inputs, including sight, sound, and scent (Juliani, et al., 2018). During evaluation, the ML agent demonstrated the ability to cut off the player's path, strategically corner the target, and dynamically use AI familiars. Notably, the agent learned to use these ground and aerial scouts specifically when the scent trail was lost, rather than relying on a fixed set of instructions (Jaderberg, et al., 2019; Lample & Chaplot, 2017).

The following graph tracks the agents progress during the training pipeline:

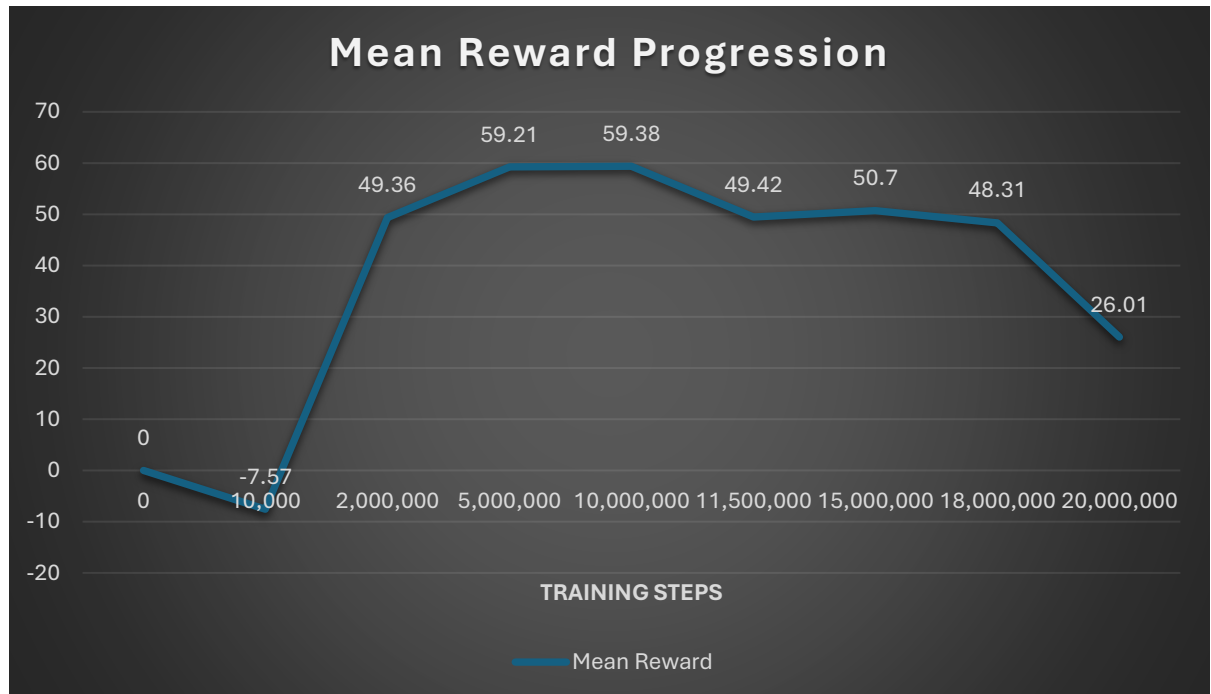


Figure 11: Mean Reward progression across the 20 million step curriculum

Analysis of the Mean Reward graph shows a rapid learning curve up to the 10 million step mark. However, a significant decline in rewards occurred during the later stages. This late-stage regression suggests that the agent began to experience training instability where trying to learn highly specific situations actually degraded its overall performance. This indicates that training beyond this 10 million step peak was counterproductive, reinforcing the idea that finding the peak reward is more important than simply increasing the amount of training.

This table below highlights how both the ML and BT agents were able to catch the prey using different navigation methods:

Performance Metric	Enemy A (RL) Avg.	Enemy B (BT) Avg.
Success Rate (capture %)	100 %	100%
Avg. Capture Time	~48 seconds	~32 seconds
Navigation Method	Physics based	NavMesh
Behaviour pattern	Stalking	Direct pursuit

Table 1: Technical Performance

The difference in capture time highlights the contrast between mathematical optimal pathing and learned physics-based steering (Khan, 2014; Mnih, et al., 2015). While the BT agent was faster at ending the game, player feedback suggested it often felt robotic because it never hesitated or took a more natural route. The ML agent's slower capture time was a direct result of its stalking logic as it would often wait for the right moment to strike rather than charging blindly at the player.

A total of 10 participants took part in the user testing phase, consisting of a mix of casual and frequent gamers to ensure a broad perspective on AI believability. Testers evaluated two distinct agents: Enemy A, the reinforcement learning Agent, and Enemy B, the behaviour tree Agent (Isbister & Nass, 2000).

Survey Metric	Scale	Enemy A (RL) Avg.	Enemy B (BT) Avg.
Predictability	1-5 1 = not predictable, 5 = very predictable	4.00	2.09
Behaviour Adaptability	1-5 1 = not adaptive, 5 = very adaptive	1.91	3.00
Tracking Effectiveness	1-5 1 = not effective, 5 = highly effective	2.91	3.73
Overall Difficulty	1-10 1 = very easy , 10 = exceedingly difficult	3.00	6.27
Natural vs Robotic	1-10 1 = very robotic, 10 = very natural	3.64	3.91

Table 2: Tester Results

Predictability and adaptability metrics are based on the criteria used by (Glavin & Madden, 2018) and (Tang, 2023).

8. Comparative Analysis

8.1 Critical Evaluation

A critical finding from this comparative analysis is the distinct shift between efficiency and unscripted in the hunting paradigms of each agent. The BT baseline executed mathematically optimal and highly efficient pathfinding around obstacles using the A* algorithm (Khan, 2014). However, this high level of efficiency resulted in rigid and highly predictable behaviour. Because the BT agent did exactly what it was programmed to do with zero wasted movement, it was easier for players to exploit. Once a player recognised the pursuit patterns of the NavMesh, they can easily manipulate the AI by path looping it around environmental obstacles (Tang, 2023).

In contrast, the Proximal Policy optimisation (PPO) ML agent operates on a completely different logic (Schulman, et al., 2017). Despite lacking native, preprogrammed pathfinding, it developed complex behaviours through millions of rounds of trial and error. The ML agent processes a massive 90-dimension observation vector, balancing vision raycasts, audio filtering, and scent strength simultaneously. This high level of sensory input resulted in a more deliberate and human like stalking phase. Because the agent was not built with simple “if then” rules, its actions are developed naturally. For example, if a player hides in bushes to mask their scent, the ML agent might lose the direct trail by continue moving toward the player’s last known trajectory. This demonstrates a predictive interception that a scripted BT cannot replicate as the ML agent adapts dynamically to evasion tactics rather than following a fixed path.

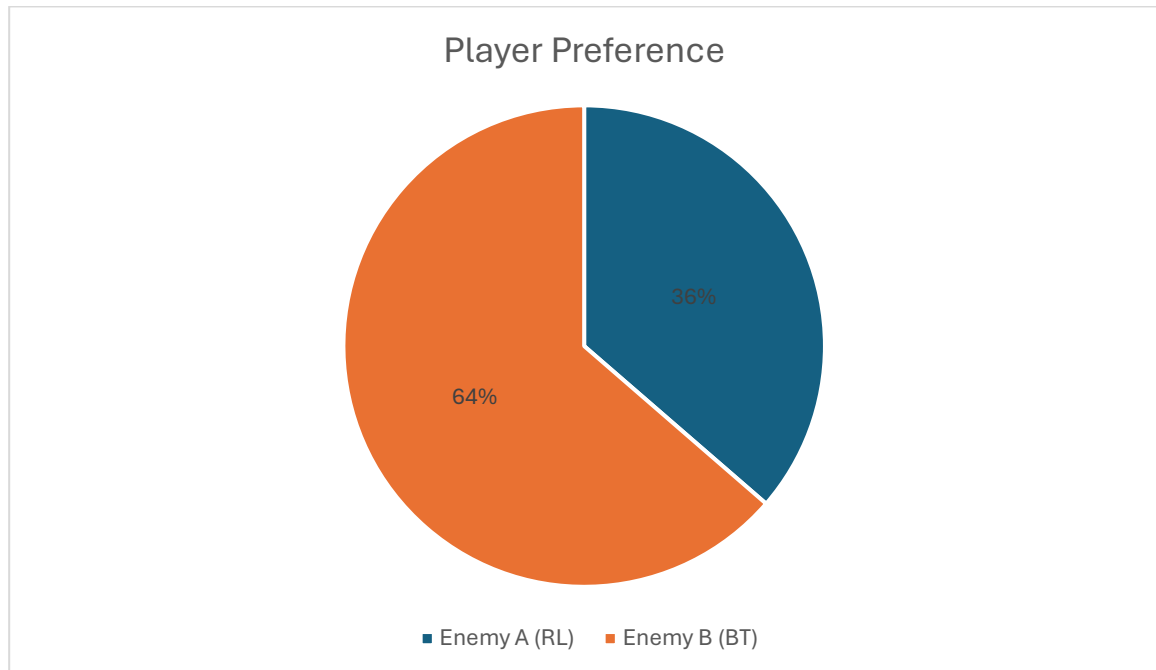


Figure 12: Player Enemy Preference

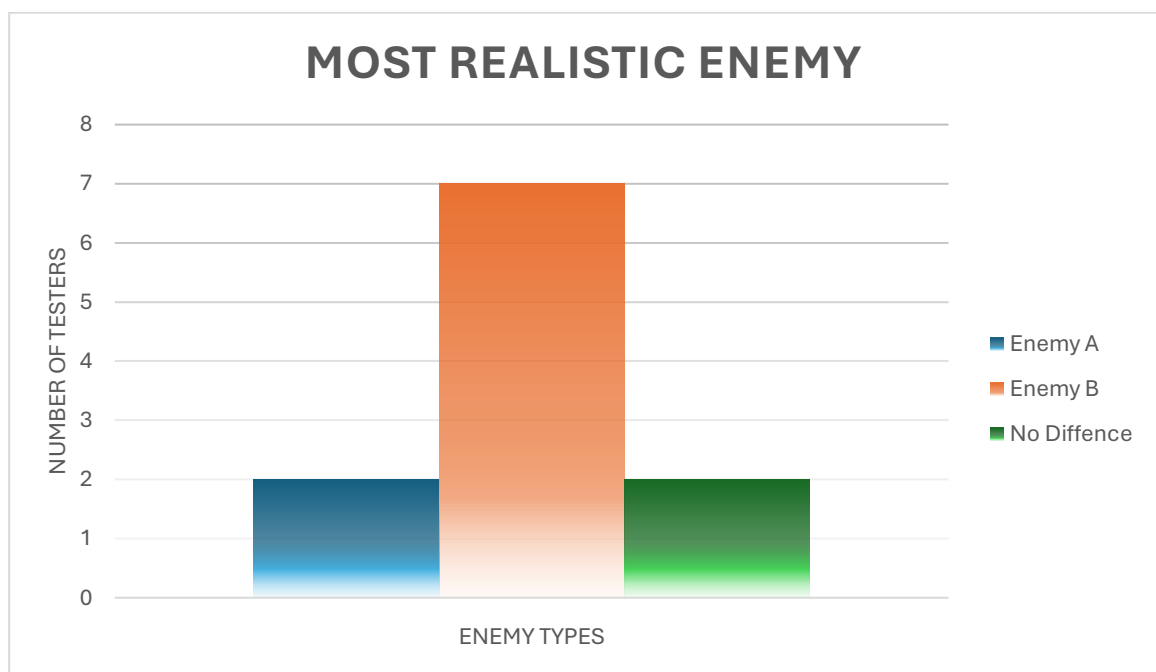


Figure 13: Perceived Realism Comparison

8.2 Analysis of Tester feedback and Exploits

The data from the testing phase indicates a clear player preference for the BT agent. Feedback suggests that because players had a much higher chance of being caught by the BT agent, they perceived it as being more realistic and intelligent. As one tester noted about the direct threat, “Enemy B was more consistent at finding me and felt like it was able to track me down faster, whereas enemy A was a lot slower.” Another tester added, “A was extremely easy got to easy 16 min before being bored, B got me in 3 min.” Some testers noticed differences in the decision-making stating, “Enemy B seemed to be a bit more cunning rather than just chasing me, but it was difficult to tell.”

While the majority of players preferred the aggressive challenge of the BT agent, feedback indicated a significantly different experience for non-gamers. For example, Tester 9, who identified as having minimal gaming experience, explicitly preferred the ML agent. Because the ML agent took longer to adapt and relied on a slower stalking method, this tester found the encounter much more accessible and less overwhelming. This highlights an important dynamic in game AI design: while mathematical optimal agents appeal to seasoned players seeking high difficulty, the developing, unscripted behaviour of an RL agent may provide a more balanced and approachable experience for casual gamers.

However, the RL agent had different strengths. The RL agent's slower, unscripted stalking approach was seen as useful for creating a more immersive and engaging tension over longer periods of time, rather than just ending the game quickly. Furthermore, the BT agent had clear technical flaws that the RL agent did not share. Testers highlighted this by saying, “ They both seem to be similar, except B is Buggy (It gets stuck).” Another player mentioned, “Once hit with a rock, they would break and stand still for a long period of time,” while another confirmed, “Enemy B got stuck on the ground for a few seconds while chasing me.”

A key factor in the player preference for the BT was the time required for the RL agent to adapt. While the BT agent's logic remained identical throughout the five-minute testing sessions, the RL agent was designed to get smarter over time as it gathered more data on a specific player's habits. However, evaluation showed that this adaptation process takes approximately 30 minutes of continuous play to become noticeable. Since the test

sessions were capped at five minutes, the RL agent did not have enough time to catch up to the player's skill level, making it feel less effective than the BT.

Furthermore, several technical exploits were discovered during the user testing phase that impacted the results. Players manipulated the environment, with one tester explaining, "I was going in circles around rocks and the AI did not like it :D." Many participants found they could avoid both agents by jumping on top of large rock obstacles. One tester stated, "I stood on a rock for 5 minutes and the enemies could not get me (clearly I'm smarter)." Because the agents were not designed to navigate vertical terrain or climb, they were unable to reach the players once this tactic was used.

In addition to navigation exploits, some participants discovered critical flaws in the combat mechanics. Multiple users reported an "infinite stun" bug where hitting the enemies with rocks caused "eternal paralysis," which completely broke the pursuit loop and prevented the agents from moving. Testers also experienced sensory disconnects with Enemy B, noting that the audio and music sometimes failed to warn them of incoming threat. These issues provided valuable data on how players naturally look for safe zones and mechanical glitches to exploit AI limitations.

The expanded feedback from all 11 testers also highlighted a distinct divide in how difficult and consistent the two agents felt. Testers widely agreed that Enemy A became predictable over time and was significantly slower, with multiple users noting it was entirely too easy to loop or evade. Enemy B, on the other hand, was described as "more consistent at finding me" and capable of sensing targets at longer ranges, creating a much higher-pressure environment. These findings suggest that while RL agent has more potential for long-term variety and immersion, the immediate challenge and mechanical pressure provided by the BT agent is more effective for short term gameplay sessions.

9. Discussion

9.1 Limitations

While the project achieved its core aims, several limitations were identified during the development process that highlight the challenges of using machine learning in game design:

- **Training instability and behavioural plateaus:** Reinforcement learning is often a fickle process. The ML agent frequently hit low reward training plateaus where it failed to progress, instead getting stuck in erratic obsessive behaviours such as spinning in circles or hugging walls. Overcoming these hurdles required heavy manual intervention and constant reward shaping. This included implementing curiosity modules (Burda, et al., 2018), rewriting the reward logic to specifically punish wall hugging (Sutton & Barto, 2018), and artificially buffing the player's speed to force the agent to learn advanced tactics. These issues show that RL is not yet a set it and forget it solution and requires significant developer oversight to stay on track.
- **Time and hardware constraints:** Due to the project's deadlines and available processing power, the neural network training was capped at 20 million steps. Although, the model had showed clear signs of convergence, additional training time on higher performance hardware could have allowed for a much larger curriculum , such as 100 million steps. This extra time might have smoothed out the agent's steering and navigation further, potentially matching the fluid movement seen in traditional NavMesh agents (Mnih, et al., 2015; Silver, et al., 2017).
- **Environmental generalisation:** The ML agent is currently an expert on its specific training map. While the environment used a randomiser script to move obstacles and distract the agent , the current policy is highly specialised to this terrain. If the agent were placed into a completely new terrain structure with different features, such as vertical cliffs or narrow indoor corridors, it would likely struggle to adapt immediately. Such a transition would require an additional fine-tuning phase to help the agent learn the new map layout. (Pan & Yang, 2010).
- **Gameplay and mechanical friction:** Beyond the AI logic, the current build suffered from minor mechanical friction that impacted the testing experience. Feedback from participants noted that the lack of a crosshair made aiming throwables difficult, and some reported visual and audio bugs, such as random white screen effects or missing audio cues during pursuits. Additionally, environmental hitbox glitches (including floating rocks, T posing animations, and a bugged escape

trigger) occasionally interfered with the player's immersion and the core gameplay loop.

9.2 Impact

The outcomes of this project offer a meaningful contribution to the practical application of AI in game development (Senanayake, 2025). Historically, Reinforcement Learning has been confined to academic research or non-playable simulations that were optimised purely for task completion (Lample & Chaplot, 2017). By successfully training a PPO agent (Schulman, et al., 2017) to operate within a real-time, player facing horror environment featuring distractions, dynamic audio and complex terrain, this project demonstrates that RL has moved beyond simple 2D tests or basic board games like go or Atari.

The project proves that RL is a viable, practical alternative to traditional AI scripting. The final artefact demonstrates that machine learning can produce NPCs that are significantly less predictable and more terrifyingly adaptable than those using fixed instructions (Jaderberg, et al., 2019). By bridging the gap between theoretical research and commercial game design, this work shows that unscripted AI can create a more rewarding and intense experience for the player.

10. Conclusion and Future Work

10.1 Conclusion

The project set out to investigate whether Reinforcement Learning could produce an adaptive and believable NPC capable of outperforming the rigid logic of traditional game AI. Through an iterative Agile methodology, a robust Unity based environment was constructed, and a PPO ML agent was trained over a rigorous 20 million step curriculum (Schulman, et al., 2017; Juliani, et al., 2018). By equipping the agent with a 90-dimension observation vector driven by vision, hearing and scent tracking, the agent successfully learned to hunt a dynamic player target while filtering out environmental noise (Jaderberg, et al., 2019). This process transformed the agent from a basic model into a predator that does not just follow a path but actively learns how to hunt.

The final head-to-head comparison against a custom A* Behaviour Tree baseline showed that the testers ultimately preferred the BT agent for short sessions. However,

the project was not a failure. It is acceptable that the RL agent did not dominate player preference, because the results highlight that the short five-minute survival format was simply the wrong type of game mode for a slow learning network. The BT agent won in immediate difficulty, but the RL agent successfully learned a highly engaging and unscripted adaptability. This creates a much more immersive experience over time, as the predator's unpredictable nature greatly enhances the overall quality of the game AI (Tang, 2023; Senanayake, 2025). Ultimately, the project proves that while the scripted AI offers quick efficiency, learned adaptability offers unique value for player engagement.

10.2 Future Work

In the future, development should focus on bringing down the performance difference between the BT and RL agent to improve player preference for the RL agent during short player sessions. Future work needs to focus on:

- **Improved Training Targets:** instead of training the RL agent for 100 million steps against a basic prey, a good idea would be to make the BT agent act as the prey. The RL agent could then train directly against the perfect mathematical pathfinding of the BT. This would teach the RL agent how to counter advanced evasion tactics much faster, lowering the required steps and making the RL agent a massive threat right from the beginning of the game.
- **Transfer learning:** Instead of retraining the AI from scratch for every new level, the existing 20 million step model could be evaluated in entirely new map terrains. This would test the agent's general intelligence and show how quickly it can adapt its hunting strategies to unfamiliar environments and different types of obstacles (Pan & Yang, 2010).
- **Multi-Agent co-op:** The familiar system could be expanded from a simple summon mechanic into a more advanced multi agent setup. By sharing knowledge and observations through a shared system, the predator and its scouts could perform coordinated pack hunting strategies. This would allow the agent to work together to flank the player from multiple angles or corner them in tight spaces (Jaderberg, et al., 2019).

- Multiplayer dynamics: Scaling the AI to handle multiple human targets would require the agent to manage a priority shifting logic. In this scenario, the predator would have to manage split attention and decide whether to commit to a single pursuit or switch targets based on proximity and the perceived vulnerability of each player (Glavin & Madden, 2018). This would test the AI's ability to make complex tactical decisions in a more chaotic environment.
- Mechanical balancing and bug resolution: immediate future work must also address the “infinite stun” exploit found during testing. The logic for throwable rocks needs to be updated to prevent players from permanently paralysing the agents. Additionally, refining the sensory warning systems to ensure audio consistently triggers when an enemy approaches, alongside fixing environmental clipping and hitbox issues around trees, will be necessary to ensure a seamless and fair testing environment.

References

Bontrager, P., Khalifa, A., Mendes, A. & Togelius, J., 2018. *Automatic playtesting for game parameter tuning via deep reinforcement learning*, s.l.: s.n.

Burda, Y. et al., 2018. *Large-Scale Study of Curiosity-Driven Learning*. Ithaca, NY, arxiv.

Datatonic, T., 2022. *Reinforcement Learning: How to Train an RL Agent from Scratch*. [Online]

Available at: <https://blog.montrealanalytics.com/reinforcement-learning-how-to-train-an-rl-agent-from-scratch-96af39a3287>

[Accessed 30 October 2025].

Glavin, F. & Madden, M. G., 2018. *Skilled Experience Catalogue: A Skill-Balancing Mechanism for Non-Player Characters using Reinforcement Learning*. s.l., s.n.

Hester, T., Vecerik, M., Pietquin, O. & al., e., 2018. *Deep Q-Learning from Demonstrations*, s.l.: s.n.

Isbister, K. & Nass, C., 2000. *How People Talk to Computers, Robots, and Other Media*. San Francisco: Morgan Kaufmann.

Jaderberg, M., Czarnecki, W., Dunning, I. & al., e., 2019. Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. *Science*, 364(6443), p. aau6249.

Juliani, A., Berges, V., Vckay, E. & al., e., 2018. *Unity ML-Agents: A toolkit for reinforcement learning and imitation learning in games*. [Online]
Available at: <https://github.com/Unity-Technologies/ml-agents>
[Accessed 27 October 2025].

Khan, A., 2014. *Pathfinding example: computed path from waypoint C to A*. [Online]
Available at: https://www.researchgate.net/figure/Pathfinding-example-computed-path-from-waypoint-C-to-A_fig5_262729027
[Accessed 30 October 2025].

Lample, G. & Chaplot, D., 2017. *Playing FPS games with deep reinforcement learning*. s.l., s.n.

MLVU, 2022. *Lecture 13: reinforcement learning*. [Online]
Available at: <https://mlvu.github.io/lecture13/>
[Accessed 29 October 2025].

Mnih, V. et al., 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540), p. 529–533.

Pan, S. & Yang, Q., 2010. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), p. 1345–1359.

Schulman, J. et al., 2017. Proximal Policy Optimization Algorithms.
arXiv:1707.06347.

Senanayake, C., 2025. *DYNAMIC NPC AI USING REINFORCEMENT LEARNING FOR AN ENHANCED GAMING EXPERIENCE*. s.l., s.n.

Silver, D., Schrittwieser, J., Simonyan, K. & al., e., 2017. Mastering the game of Go without human knowledge. *Nature*, Volume 550, p. 354–359.

Sutton, R. & Barto, A., 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Tang, Z., 2023. *Reinforcement Learning in Digital Games: An Exploration of AI in Gaming*. s.l., s.n.

Zook, A., Fruchter, E. & Riedl, M., 2019. *Automatic Playtesting for Game Parameter Tuning via Active Learning*, s.l.: s.n.

List of Assets used in implementation

- ML agent support - unity - <https://docs.unity3d.com/Packages/com.unity.ml-agents@3.0/manual/index.html>
- ML agent support - LudicWorlds - <https://www.youtube.com/@LudicWorlds>
- Font Type - Brian Kent - <http://aenigma.cellosoft.com/>
- Unity assets - ALP - <https://assetstore.unity.com/publishers/37465>
- Unity assets - Nobiax - <https://assetstore.unity.com/publishers/4986>
- Unity assets - Wello Soft - <https://assetstore.unity.com/publishers/11882>
- player model - Tremolo_1404_ - https://sketchfab.com/Tremolo_1404_
- predator model - JonnyMANSON - <https://sketchfab.com/JonnyMANSON>
- creature model - TRITH - <https://sketchfab.com/willisjonathan689>
- creature model - gavinpgamer1 - <https://sketchfab.com/gavinpgamer1>
- creature model - CataRackta - <https://sketchfab.com/CataRackta>
- animations - mixamo - <https://www.mixamo.com>
- Audio source - freesound_community - https://pixabay.com/users/freesound_community-46691455/
- Audio source - dffdv - <https://pixabay.com/users/dffdv-42492812/>
- Audio source - Yodguard - <https://pixabay.com/users/yodguard-12455005/>
- Audio source - Flashworks - <https://pixabay.com/users/24881608/>

- Audio source - AlesiaDavina - <https://pixabay.com/users/alesiadavina-32608554/>
- Audio source - DRAGON-STUDIO - <https://pixabay.com/users/dragon-studio-38165424/>
- Audio music - meak363 - <https://pixabay.com/users/meak363-43664287/>
- Audio music - INPLUSMUSIC - <https://pixabay.com/users/lnplusmusic-47631836/>

Appendices

A. Training Configuration (YAML)

```
behaviors:
  Predator:
    trainer_type: ppo

    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 1.5e-4
      beta: 5.0e-4
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
      beta_schedule: constant
      epsilon_schedule: linear

    network_settings:
      normalize: true
      hidden_units: 256
      num_layers: 3
      vis_encode_type: simple

    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
      curiosity:
        gamma: 0.99
        strength: 0.005
        network_settings:
          hidden_units: 256
          num_layers: 2
          learning_rate: 1.5e-4

    keep_checkpoints: 5
    max_steps: 2000000
    time_horizon: 128
    summary_freq: 10000
```

Figure 14: YAML

B. Reinforcement Learning Agent Logic

```

public override void CollectObservations(VectorSensor sensor)
{
    float dist = Vector3.Distance(transform.position, player.position);

    // player direction and distance
    sensor.AddObservation(dist / 50f); // 1 obs
    sensor.AddObservation(transform.InverseTransformDirection((player.position
- transform.position).normalized)); // 3 obs
    sensor.AddObservation(predatorController.velocity / 10f); // 3 obs

    // vision and recognition state
    bool visible = CheckLineOfSight();
    sensor.AddObservation(visible ? 1f : 0f); // 1 obs
    sensor.AddObservation(preyIsRecognised ? 1f : visionRecognitionTimer /
recognitionThreshold); // 1 obs

    // predator state
    sensor.AddObservation(isCrouching ? 1f : 0f); // 1 obs
    sensor.AddObservation(Mathf.Clamp01(timeSinceSeen / 15f)); // 1 obs
    sensor.AddObservation(Mathf.Clamp01(familiarCooldownRemaining /
familiarCooldown)); // 1 obs
    sensor.AddObservation(hasFoundTrailThisEpisode ? 1f : 0f); // 1 obs

    // prey state
    sensor.AddObservation(preyAi != null && preyAi.isSprinting ? 1f : 0f); // 1
obs
    sensor.AddObservation(preyAi != null && preyAi.isCrouching ? 1f : 0f); // 1
obs
    sensor.AddObservation(preyAi != null && preyAi.isExposed ? 1f : 0f); // 1
obs
    sensor.AddObservation(preyAi != null && preyAi.inSoftObj ? 1f : 0f); // 1
obs

    // sound
    Vector3 soundDir = SoundEmitter.LastSoundPos - transform.position;
    sensor.AddObservation(soundDir.sqrMagnitude < 0.01f ? Vector3.zero :
transform.InverseTransformDirection(soundDir.normalized)); // 3 obs
    sensor.AddObservation(Mathf.Clamp01(SoundEmitter.LastSoundVolume)); // 1
obs

    // recent scent trail
    if (preyTrail != null && preyTrail.MainTrail != null)
    {
        int count = preyTrail.MainTrail.Count;
        for (int i = 0; i < 3; i++)
        {
            int index = count - 1 - i;
            if (index < 0)
            {
                sensor.AddObservation(Vector3.zero); // 3 obs
                sensor.AddObservation(0f); // 1 obs
            }
            else
            {
                Vector3 scentDir = preyTrail.MainTrail[index] -
transform.position;
                sensor.AddObservation(transform.InverseTransformDirection(scentDir.normalized));
                sensor.AddObservation(Mathf.Clamp01(scentDir.magnitude /
rayDistance));
            }
        }
    }
}

public override void OnActionReceived(ActionBuffers actions)
{
    if (stunTimer > 0f)
    {
        stunTimer -= Time.fixedDeltaTime;
        predatorController.Move(new Vector3(0, -2f * Time.fixedDeltaTime, 0));
        if (animator != null)
        {
            animator.SetBool("isMoving", false);
            animator.SetBool("isSprinting", false);
            animator.SetFloat("moveSpeed", 0f);
        }
        return;
    }

    episodeTimer += Time.fixedDeltaTime;

    // headstart
    if (episodeTimer < headstartDuration)
    {
        predatorController.Move(new Vector3(0, -2f * Time.fixedDeltaTime, 0));
        if (animator != null)
        {
            animator.SetBool("isMoving", false);
            animator.SetBool("isSprinting", false);
            animator.SetFloat("moveSpeed", 0f);
        }
        return;
    }

    float forward = Mathf.Clamp(actions.ContinuousActions[0], 0f, 1f); // move
and backwards but stops it from moonwalking
    // float strafe = Mathf.Clamp(actions.ContinuousActions[1], -0.2f, 0.2f); //
move left/right and limiting the strafing
    float turn = Mathf.Clamp(actions.ContinuousActions[2], -1f, 1f); // rotate

    // smooth turning no jittering
    smoothedTurn = Mathf.Lerp(smoothedTurn, turn, Time.fixedDeltaTime * 10f);
    smoothedForward = Mathf.Lerp(smoothedForward, forward, Time.fixedDeltaTime *
8f);

    if (tauntTimer > 0f)
    {
        tauntTimer -= Time.fixedDeltaTime;

        forward = 0f;
        turn = 0f;
        smoothedTurn = 0f;
    }

    // recognition state
    float recognitionModulator = 1f;
    bool hasLOS = CheckLineOfSight();

    if (hasLOS)
    {
        continuousLosTimer += Time.fixedDeltaTime;
        timeSinceLastLos = 0f;
    }
}

```

Figure 15: ML Agent - Observation and Reward System

C. Behaviour Tree Baseline Logic

```

void BuildTree()
{
    // tree is evaluated top to bottom, top has higher priority
    rootNode = new BTSelector(new List<BTNode>)
    {
        // Priority 1: headstart
        new BTSequence(new List<BTNode>
        {
            new BTCondition(() => episodeTimer < headstartDuration),
            new BTAction(DoHeadstart)
        }),
        // Priority 2: taunt
        new BTSequence(new List<BTNode>
        {
            new BTCondition(() => tauntTimer > 0f),
            new BTAction(DoTaunt)
        }),
        // Priority 3: strik
        new BTSequence(new List<BTNode>
        {
            new BTCondition(CheckLineOfSight),
            new BTCondition(() => preyIsRecognised),
            new BTAction(DoStrike)
        }),
        // Priority 4: stalk
        new BTSequence(new List<BTNode>
        {
            new BTCondition(CheckLineOfSight),
            new BTAction(DoStalk)
        }),
        // Priority 5: scent tracking
        new BTSequence(new List<BTNode>
        {
            new BTCondition(HasScentTrail),
            new BTAction(DoTrackScent)
        }),
        // Priority 6: sound tracking
        new BTSequence(new List<BTNode>
        {
            new BTCondition(HasRecentSound),
            new BTAction(DoTrackSound)
        }),
        // Priority 7: summon familiar
        new BTSequence(new List<BTNode>
        {
            new BTCondition(CanSummonFamiliar),
            new BTAction(DoSummonFamiliar)
        }),
        // Priority 8: wander
        new BTAction(DoWander)
    });
}

// stand still during headstart
NodeState DoHeadstart()
{
    CurrentBehaviour = "HEADSTART";
    navAgent.ResetPath();
    navAgent.speed = 0f;
    return NodeState.Running;
}

// freeze a little after recognition
NodeState DoTaunt()
{
    CurrentBehaviour = "TAUNT";
    tauntTimer -= Time.deltaTime;

    if (tauntTimer == tauntDuration)
    {
        animator.SetTrigger("onTaunt");
    }

    navAgent.ResetPath();
    navAgent.speed = 0f;

    // look at player while taunting
    Vector3 lookDir = (player.position - transform.position);
    lookDir.y = 0;
    if (lookDir.sqrMagnitude > 0.01f)
        transform.rotation = Quaternion.RotateTowards(transform.rotation,
        Quaternion.LookRotation(lookDir), 250f * Time.deltaTime);
    return NodeState.Running;
}

// chase at boosted speed
NodeState DoStrike()
{
    CurrentBehaviour = "STRIKING";
    navAgent.speed = baseSpeed * recognitionSpeedBoost;
    navAgent.SetDestination(player.position);
    return NodeState.Running;
}

// slowly approach while recognitionTimer builds
NodeState DoStalk()
{
    CurrentBehaviour = "STALKING";
    float ramp = 0.2f + (visionRecognitionTimer / recognitionThreshold) * 0.8f;
    navAgent.speed = baseSpeed * ramp;
    navAgent.SetDestination(player.position);
    return NodeState.Running;
}

// follow most recent scent trail
NodeState DoTrackScent()
{
    CurrentBehaviour = "TRACK_SCENT";
    navAgent.speed = baseSpeed;

    Vector3 freshest = preyTrail.MainTrail[preyTrail.MainTrail.Count - 1];
    navAgent.SetDestination(freshest);
    return NodeState.Running;
}

// move towards last heard sound source

```

Figure 16: BT Agent – BT Structure and Nodes

D. Risk Assessment and Mitigation

Potential Risk	Likelihood	Impact	Mitigation Strategy
Late-Stage User Testing: short window for data collection and analysis.	High	High	Used a custom Debug UI for live observations and paired this with a Likert scale questionnaire to collect both player and performance feedback.
Platform Compatibility (windows only):	Medium	Low	Specified Windows as the target platform in requirements, ensured

The build is limited to the windows platform.			all recruited testers have access to compatible hardware for the evaluation.
Stagnation in RL Training: The agent stops exploring and sticks to suboptimal behaviours.	High	Medium	Implemented curiosity driven rewards and set a constant Beta(randomness) value in predator.yaml. These forced the agent to try new actions instead of repeating.

Table 3 : Risk Assessment